Nguyên lý triển khai Backend Service

Từ OpenAPI definitions đến Flask service có tích hợp database — nguyên lý, cấu trúc project, ORM, migration và triển khai Docker cho backend API hướng tới triển khai thực tế.

#Flask #SQLAlchemy #Docker

MỤC TIÊU

Triển khai backend API từ OpenAPI definitions

Đọc được một file OpenAPI 3.0 và lựa chọn cách tiếp cận (Design-First hoặc Code-First) để generate hoặc implement các route tương ứng bằng Flask.

Tích hợp backend với database

Khai báo Model với SQLAlchemy 2.x, định nghĩa Schema với Marshmallow, viết Resource xử lý CRUD, tổ chức migration bằng Flask-Migrate.

NỘI DUNG BUỔI HỌC

Phần 1. Application Factory, Blueprints, OpenAPI với Flask

Phần 2. SQLAlchemy 2.x ORM, Marshmallow Schema, Migration

Phần 3. Error handling, logging, config, Docker, WSGI

Backend service là gì, nằm ở đâu trong hệ thống?



MỘT BACKEND SERVICE PHẢI

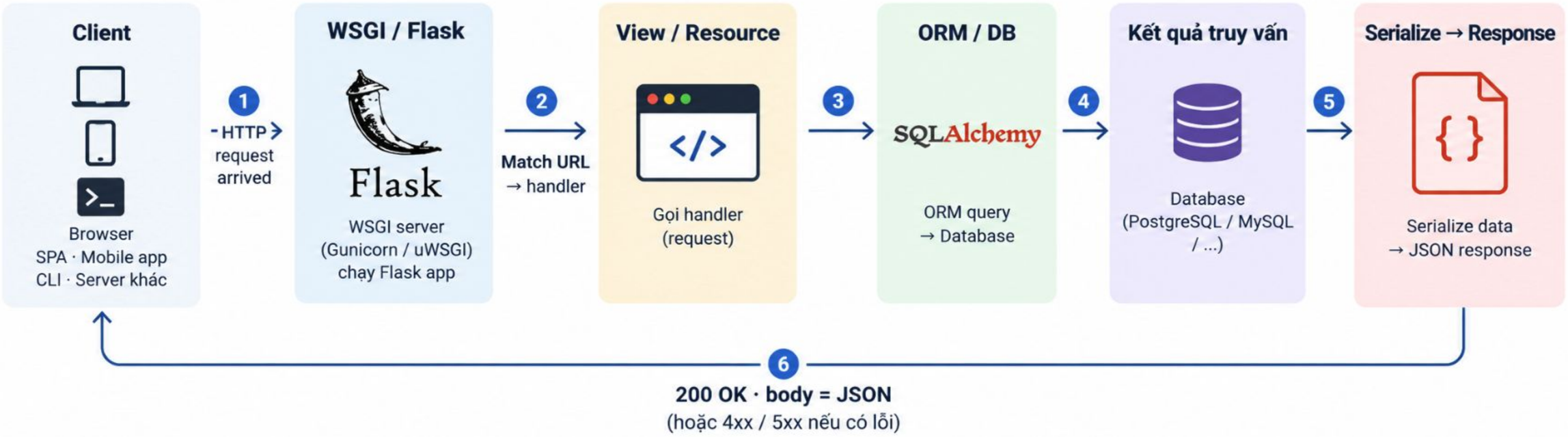
01. **Parse & validate** request hợp lệ trước khi xử lý.

02. **Áp dụng logic** nghiệp vụ có kiểm thử.

03. **Truy cập storage** qua abstraction (ORM / repository).

04. **Trả response** có status code & shape nhất quán.

Một HTTP request đi qua Flask như thế nào?



Buổi này tập trung vào ② ③ ④ ⑤ — cách handler đăng ký, cách parse input, cách gọi DB, cách serialize output.

03 · NGUYÊN LÝ

Bảy nguyên lý thiết kế backend không nên bỏ qua

Tổng hợp các thực hành thiết kế và vận hành backend; không phải trích dẫn nguyên văn từ một nguồn.

01

Stateless request

Mỗi request phải chứa đủ thông tin để xử lý. Không giữ session trên server.

02

Validate ở ngõ vào

Không bao giờ tin input. Reject ngay request không hợp lệ với 4xx rõ nguyên nhân.

03

Failure là bình thường

Default fail-recovery, idempotent cho state-changing endpoint, log & metric mọi lỗi.

04

Layered service

Tách transport · service · persistence. Mỗi lớp thay đổi vì lý do khác nhau.

05

Config qua env

Mọi tham số (DB URL, secret, mode) đi qua biến môi trường, không hard-code trong source.

06

Log có cấu trúc

Một dòng JSON một sự kiện, có request_id, level, latency_ms — query được bằng ELK / Loki.

07

12-factor & health check

Build một artifact, deploy mọi nơi. Có endpoint /healthz (liveness) và /ready (readiness) để orchestrator biết lúc nào gửi traffic và lúc nào restart.

Code-First và Design-First khác nhau ở đâu?

Cả hai đều dùng OpenAPI 3.0 làm "hợp đồng" — câu hỏi là: viết spec trước, hay viết code trước?

CÁCH A

Code-First

Viết handler trước, OpenAPI sinh sau từ code.

`code (Flask)` → `spec / doc` → `Swagger UI`

Phù hợp khi:

- Team quen Python, ít về API design
- Prototype nhanh, MVP, team 1-2 người
- Schema cho mỗi resource thay đổi liên tục

VD:

`flask-smorest` · `flask-apispec`

CÁCH B

Design-First

Soạn OpenAPI spec trước rồi ánh xạ operation đến handler.

`openapi.yaml` → `routes + schema` → `handler logic`

Phù hợp khi:

- API là sản phẩm public, nhiều consumer
- Spec cần review trước khi build backend
- Multi-team, có mobile/web cùng dùng 1 spec

VD:

connexion; flask-openapi3 (lựa chọn khác)

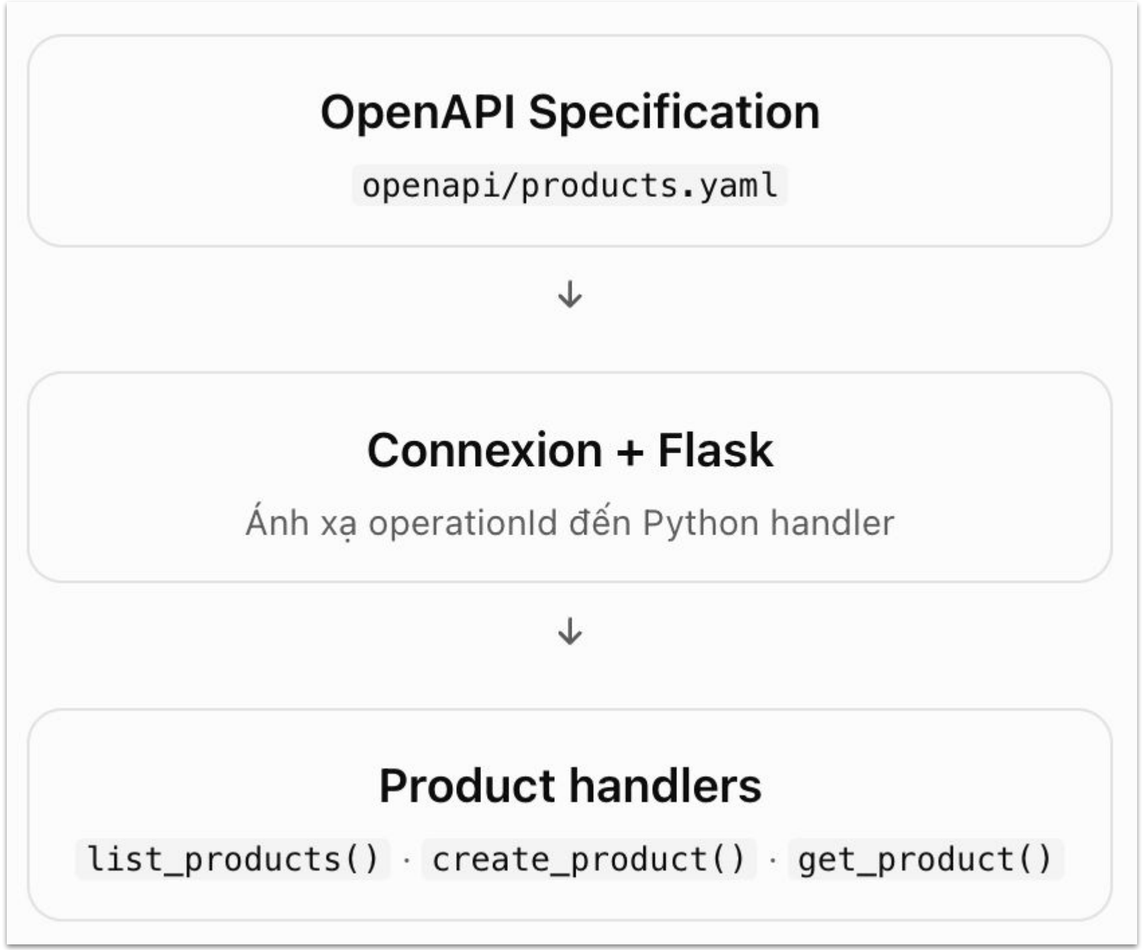
Viết OpenAPI spec trước — connexion tự resolve thành Flask routes

① Spec openapi/products.yaml ▸ ② Handler api/products.py ▸ ③ app.py wire together

openapi/products.yaml — hợp đồng mô tả input và response

```yaml
# OpenAPI 3.0
openapi: 3.0.3
info: { title: Product API, version: 1.0.0 }
paths:
  /products:
    get:
      operationId: products.list_products
      responses:
        '200': { description: OK }
    post:
      operationId: products.create_product
      requestBody:
        required: true
        content:
          application/json:
            schema:
              { $ref: '#/components/schemas/ProductIn' }
      responses:
        '201': { description: Created }
        '422': { description: Validation }
  /products/{id}:
    get:
      operationId: products.get_product
      parameters:
        - { name: id, in: path, required: true,
            schema: { type: integer } }
      responses:
        '200': { description: OK }
        '404': { description: Not Found }
```

Viết OpenAPI spec trước — connexion tự resolve thành Flask routes




```yaml
paths:
  /products:
    get:
      operationId: products.list_products
      responses:
        '200':
          description: OK
```

```python
def list_products():
    rows = db.session.execute(
        db.select(Product)
    ).scalars().all()

    return [p.to_dict() for p in rows], 200
```

Viết OpenAPI spec trước — connexion tự resolve thành Flask routes

① Spec openapi/products.yaml ▸ ② Handler api/products.py ▸ ③ app.py wire together

api/products.py — function name trùng operationId

```python
def list_products():
    rows = db.session.execute(
        db.select(Product)).scalars().all()
    return [p.to_dict() for p in rows], 200

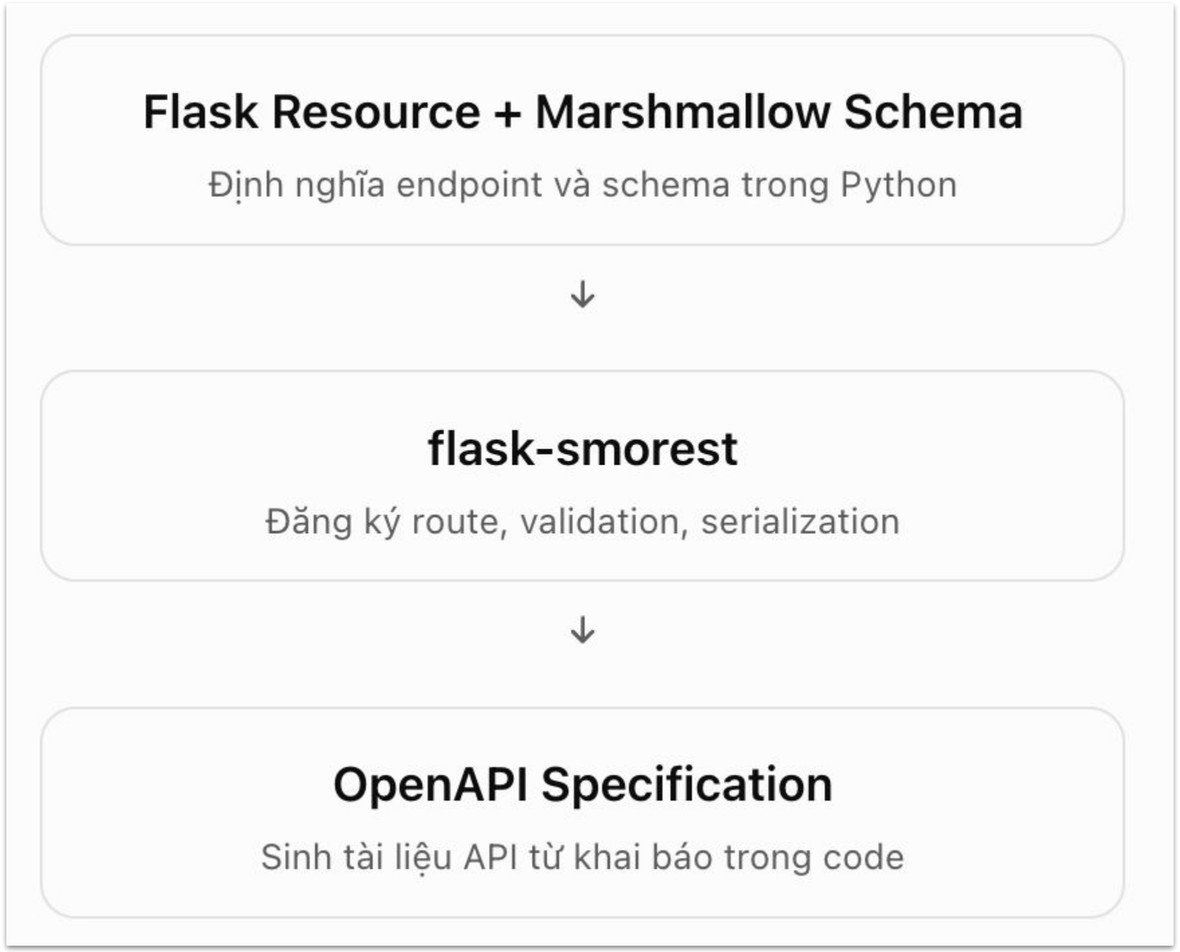
def create_product(body):
    p = Product(**body)
    db.session.add(p)
    db.session.commit()
    return p.to_dict(), 201

def get_product(id):
    p = db.get_or_404(Product, id)
    return p.to_dict(), 200
```

app.py

```python
import connexion
app = connexion.FlaskApp(
    __name__, specification_dir="openapi/")
app.add_api("products.yaml",
            validate_responses=True)
# Đường dẫn UI/spec phụ thuộc cấu hình Connexion
```

flask-smorest: viết resource, OpenAPI tự sinh



flask-smorest: viết resource, OpenAPI tự sinh

api/products.py

```python
from flask.views import MethodView
from flask_smorest import Blueprint, abort
from marshmallow import Schema, fields
from .schemas import ProductSchema, ProductInSchema
from .models import Product
from ..extensions import db
blp = Blueprint("products", __name__, url_prefix="/products")

class ProductList(MethodView):
    @blp.arguments(ProductInSchema)  # validate POST body
    @blp.response(201, ProductSchema)
    def post(self, data):
        p = Product(**data)
        db.session.add(p); db.session.commit()
        return p

    @blp.response(200, ProductSchema(many=True))
    def get(self):
        return db.session.execute(db.select(Product)).scalars().all()

# Register
blp.add_url_rule("", view_func=ProductList.as_view("product_list"))
# Cần khai báo cấu hình API_SPEC_OPTIONS và UI path
```

Ví dụ dùng MethodView với flask-smorest — cho phép gom GET/POST cùng prefix, dễ test từng method.

Khi nào chọn connexion, khi nào chọn flask-smorest?

Tiêu chí	connexion	flask-smorest
Nguồn gốc hợp đồng	File OpenAPI YAML/JSON, có thể cùng repo	Marshmallow Schema ngay trong source
Validation library	Tự validate theo OpenAPI 3.0 schema	Marshmallow Schema (mạnh, tùy biến)
Phù hợp nhất với	API là sản phẩm public, multi-team	Internal tool, MVP, schema tiến hóa nhanh
Phong cách handler	Hàm thường, tên trùng operationId	Class kế thừa MethodView
Trạng thái bảo trì	Kiểm tra phiên bản/compatibility trước khi dùng	Kiểm tra version và tài liệu chính thức
Khuyến nghị	Dùng lại spec đã xây dựng ở buổi OpenAPI — chuyển sang connexion để thấy liên kết.	Lab 1 của buổi 7 — dùng kết hợp cùng SQLAlchemy để thấy schema validation/serialization và tài liệu OpenAPI.

> **Quy tắc ngón tay cái:** team nhỏ, code-first để tăng tốc; team lớn hoặc API public, design-first để thống nhất hợp đồng trước khi build.

Application Factory: tại sao `create_app()` thay vì `app = Flask(...)`?

Lỗi thường gặp

```
# app.py — top level
from flask import Flask
app = Flask(__name__)
app.config["DB_URL"] = "sqlite:///prod.db"
# gài cứng!
db.init_app(app)
import views  # circular import
```

Cách viết đúng

```
# app/factory.py
def create_app(config=None):
    app = Flask(__name__)
    app.config.from_object(
        config or DefaultConfig)
    db.init_app(app)
    migrate.init_app(app, db)
    app.register_blueprint(products_bp)
    return app
```

BA LỢI ÍCH CỦA FACTORY

1. Cấu hình khác nhau theo môi trường

Test tạo app với SQLite in-memory; staging với Postgres schema tạm; production với connection pool.

2. Giảm nguy cơ import vòng

Tổ chức import và khởi tạo extension rõ ràng; vẫn phải quản lý import vòng.

3. Test dễ

Mỗi test có thể tạo app với cấu hình riêng, hạn chế phụ thuộc lẫn nhau. Hỗ trợ pytest fixture app.

Một cấu trúc project Flask tham khảo

```
product-api/
├── pyproject.toml
├── .env.example
├── openapi/
│   └── products.yaml
├── migrations/
│   ├── env.py
│   └── versions/
├── src/
│   └── product_api/
│       ├── __init__.py
│       ├── factory.py
│       ├── config.py
│       ├── extensions.py
│       ├── api/
│       │   ├── __init__.py
│       │   ├── products.py
│       │   └── schemas.py
│       └── models/
│           ├── __init__.py
│           └── product.py
├── tests/
│   ├── conftest.py
│   └── test_products.py
└── wsgi.py
```

- src/product_api/ — đặt code trong src/ tránh vô tình import từ root.
- Mỗi resource có file api/<name>.py + schemas tương ứng.
- extensions.py khởi tạo db/migrate một lần ở module-level.
- wsgi.py: app = create_app().
- .env.example liệt kê mọi biến môi trường cần thiết.
- migrations/ do Alembic quản lý; phải đọc và có thể chỉnh revision được sinh.

Extension và Blueprint — tổ chức ứng dụng Flask với Application Factory

KHỐI 1 — EXTENSION

Extension

```
# extensions.py
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate
db = SQLAlchemy()
migrate = Migrate()
```

Khai báo ở *module-level*, không kèm app; factory sẽ gọi `db.init_app(app)`.

KHỐI 2 — BLUEPRINT

Blueprint

```
# api/products.py
from ..extensions import db
products_bp = Blueprint(
    "products", __name__,
    url_prefix="/api/v1/products")
```

Một blueprint *biết* extension nhưng *chưa* gắn vào app. Factory gọi `app.register_blueprint(products_bp)`.

VÒNG ĐỜI CỦA MỘT REQUEST

① Request đến	② Match URL	③ Handler chạy	④ ORM query DB	⑤ Response
WSGI server nhận `GET /products`	Flask resolve → handler	Business logic `list_products()`	SQLAlchemy qua session	JSON 200/4xx/5xx

Factory + Blueprint + Extension

extensions.py

```
# Module-level proxy
# chờ được init_app()
from flask_sqlalchemy import (
    SQLAlchemy)
from flask_migrate import (
    Migrate)
db = SQLAlchemy()
migrate = Migrate()
# Tách khỏi app giúp:
# - nhiều app cùng dùng
# - test tạo app riêng
# không leak config
```

factory.py

```
from flask import Flask
from .extensions import (
    db, migrate)
from .api.products import (
    products_bp)
from .config import (
    DefaultConfig)
# Gọi được nhiều lần
def create_app(config=None):
    app = Flask(__name__)
    app.config.from_object(
        config or DefaultConfig)
    db.init_app(app)
    migrate.init_app(app, db)
    app.register_blueprint(
        products_bp)
    return app
```

api/products.py

```
from flask import Blueprint
from ..extensions import db
from ..models import Product
products_bp = Blueprint(
    "products", __name__,
    url_prefix="/api/v1/products",
)
# GET /api/v1/products
@products_bp.get("")
def list_products():
    rows = db.session.execute(
        db.select(Product)
    ).scalars().all()
    return [p.to_dict()
            for p in rows], 200
```

Khi nào nên dùng ORM thay cho SQL thuần?

CÁCH 1 — RAW SQL qua cursor

Linh hoạt nhưng cần quản lý truy vấn cẩn thận

```
# Endpoint tạo sản phẩm
cursor.execute(
    """INSERT INTO products
       (sku, name, price_cents, created_at)
       VALUES (?, ?, ?, ?)""",
    (sku, name, price, now))
conn.commit()
# ① SQL injection nếu nối chuỗi
# ② đổi schema phải sửa N chỗ
# ③ không có type-check lúc code
# ④ test factory phải bind
#    với từng câu SQL cụ thể
```

CÁCH 2 — ORM (SQLAlchemy 2.x)

Ánh xạ đối tượng, hỗ trợ tham số hóa

```
# Endpoint tạo sản phẩm
product = Product(
    sku=sku, name=name,
    price_cents=price,
)
db.session.add(product)
db.session.commit()
# ① SQLAlchemy tạo truy vấn tham số hóa
# ② giảm lặp, vẫn cần migration và sửa truy vấn
# ③ IDE + mypy check được kiểu
# ④ model-based test, dễ fake
```

BỐN LỢI ÍCH CỤ THỂ TỪ ORM

Mapping quan hệ

Khai báo `relationship()`; ORM tự lo JOIN & lazy/eager load.

Đa dialect

Viết một lần chạy trên PostgreSQL, MySQL, SQLite — khả chuyển tốt, nhưng cần kiểm tra khác biệt dialect.

Khai báo Model theo phong cách typed của SQLAlchemy 2.x

src/product_api/models/product.py

```python
from datetime import datetime
from typing import Optional
from sqlalchemy import String, Integer, Boolean, DateTime, func
from sqlalchemy.orm import Mapped, mapped_column
from ..extensions import db

class Product(db.Model):
    __tablename__ = "products"
    id: Mapped[int] = mapped_column(primary_key=True)
    sku: Mapped[str] = mapped_column(
        String(32), unique=True, index=True, nullable=False)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[Optional[str]] = mapped_column(String(2000))
    price_cents: Mapped[int] = mapped_column(Integer, nullable=False)
    qty_in_stock: Mapped[int] = mapped_column(Integer, default=0,
        nullable=False)
    archived: Mapped[bool] = mapped_column(
        Boolean, default=False, index=True)
    created_at: Mapped[datetime] = mapped_column(
        DateTime, server_default=func.now(), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(
        DateTime, server_default=func.now(),
        onupdate=func.now(), nullable=False)
```

ĐIỂM CẦN NHỚ

- `Mapped[T]` cho type-checker.
- `server_default` để DB sinh giá trị.
- `unique=True` + `index=True` cho cột hay truy vấn.
- `Optional[T]` = nullable.
- `onupdate=func.now()` cho cột updated_at.
- Dùng `Boolean` thay `Integer` cho cột 2 giá trị.

Tương thích Flask-SQLAlchemy 3.x & SQLAlchemy 2.0+. Kiểu Optional[T] cần import từ typing (không phải sqlalchemy).

Model, Schema, Resource

PERSISTENCE

Model

mô tả cấu trúc và quan hệ của dữ liệu được lưu trong database

- Có PK, FK, relationship
- Biết về cột và bảng khác
- *Không biết* HTTP
- Thao tác qua SQLAlchemy Session

→

SERIALIZATION

Schema

Định nghĩa **shape** của request và response.

- Validate input
- Serialize model → JSON
- Không truy cập DB
- Dùng Marshmallow

→

TRANSPORT

Resource

Xử lý **một request** — parse input, gọi repo, render output.

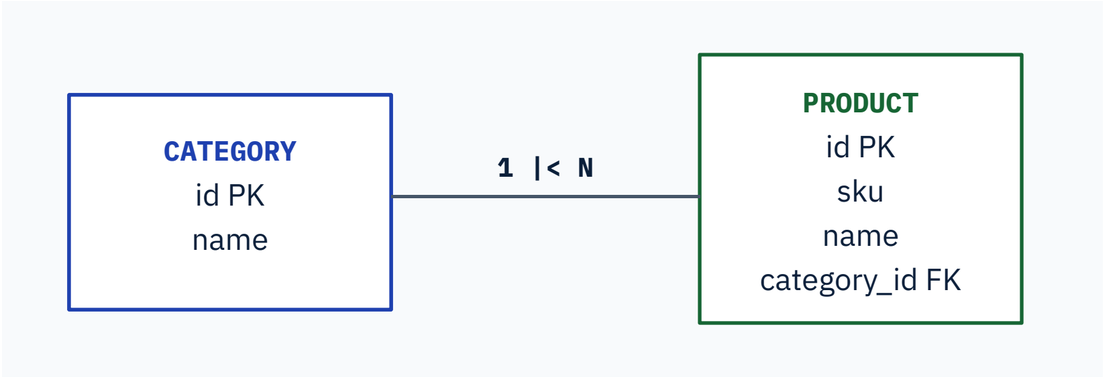
- Nhận JSON từ HTTP request.
- Dùng ProductIn để validate input.
- Tạo đối tượng Product và lưu vào database thông qua SQLAlchemy.
- Dùng ProductOut để serialize dữ liệu và trả về HTTP 201 Created.

schemas.py — 2 schema cho resource

```
from marshmallow import Schema, fields, validate
class ProductIn(Schema):
    sku = fields.Str(required=True,
        validate=validate.Length(min=3, max=32))
    name = fields.Str(required=True)
    price_cents = fields.Int(required=True,
        validate=validate.Range(min=0))
class ProductOut(Schema):
    id = fields.Int(dump_only=True)
    sku = fields.Str()
    name = fields.Str()
    price_cents = fields.Int()
```

- **Model:** Dữ liệu được lưu trữ như thế nào?
- **Schema:** Dữ liệu nào hợp lệ và được phép trao đổi qua API?
- **Resource**: HTTP request được tiếp nhận, điều phối và trả kết quả như thế nào?

Quan hệ 1-N và bẫy N+1 query — lỗi phổ biến nhất của sinh viên mới



Một Category có nhiều Product;

Product trỏ về Category qua category_id.

CÁCH SAI — phát 1 + N query

```
# Tối đa 101 query nếu đủ 100 category có lazy load
categories = db.session.execute(
    db.select(Category).limit(100)).scalars().all()
for c in categories:
    count = len(c.products)  # lazy load mỗi vòng!
# Logs:
SELECT * FROM categories LIMIT 100;  # 1 query
SELECT * FROM products WHERE cat_id=1;  # 100 query …
SELECT * FROM products WHERE cat_id=2;
```

HƯỚNG SỬA — eager load

```
# Thường 2 thay vì tối đa 101 query
from sqlalchemy.orm import selectinload
stmt = db.select(Category).options(
    selectinload(Category.products)
).limit(100)
cats = db.session.execute(
    stmt).scalars().all()
# Minh họa: 2 query khi không cần chia batch
SELECT * FROM categories LIMIT 100;
SELECT * FROM products
WHERE cat_id IN (...);
```

joinedload vs selectinload

CÁCH 1 joinedload

Phát **một query** có LEFT OUTER JOIN.

```
db.session.execute(
    db.select(Category).options(
        joinedload(Category.products)
    )
).scalars().unique().all()
# unique() — vì JOIN sinh trùng hàng
```

Dùng khi:
Thường hiệu quả với quan hệ to-one; to-many cần cân nhắc số dòng JOIN.

Tránh khi:
Quan hệ to-many lớn có thể gây nhân số dòng kết quả.

CÁCH 2 selectinload

Thường phát thêm truy vấn SELECT ... IN; có thể chia batch.

```
db.session.execute(
    db.select(Category).options(
        selectinload(
            Category.products)
            # Không có tham số limit
    )
).scalars().all()
```

Dùng khi:
Quan hệ *one-to-many* và *many-to-many*. Thường là lựa chọn an toàn nhất trong production.

Đã có sẵn:
Từ SQLAlchemy 1.4; không cần unique().

Mặc định dùng **selectinload**; chỉ dùng joinedload khi biết quan hệ là to-one. Đặt hàm load relationship *tại nơi tạo query*, không để lazy load xảy ra trong template/serializer. Bật SQL log trong profile code path quan trọng — đếm số query, không đoán.

Migration với Flask-Migrate — workflow 4 bước chuẩn production

01

Sửa Model

Thêm/sửa field trong class. Update nếu FK đổi.

02

Sinh revision

`flask db migrate -m "add sku idx"`

03

Đọc file revision

Alembic không diễn dịch đúng mọi thay đổi (enum, server_default).

04

Áp dụng & commit

`flask db upgrade`; review & git commit file trong `migrations/versions/`.

shell session mẫu

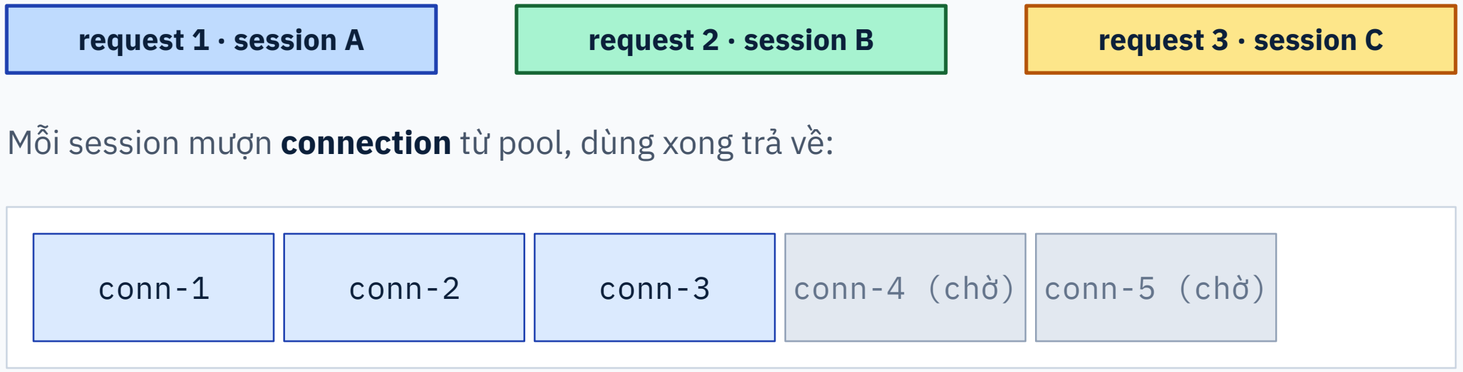
```
$ flask db init  # lần đầu
$ flask db migrate -m "add sku idx"
INFO [alembic] detected added index 'ix_products_sku'
Generating migrations/versions/abc123_add_sku_idx.py
$ cat migrations/versions/abc123_add_sku_idx.py
def upgrade():
    op.create_index('ix_products_sku', 'products',
                    ['sku'], unique=True)
def downgrade():
    op.drop_index('ix_products_sku', table_name='products')
$ flask db upgrade
INFO [alembic] Running upgrade -> abc123
```

autogenerate CẦN KIỂM TRA KỸ

- Đổi tên cột / bảng — Alembic hiểu là drop+add, mất dữ liệu.
- Constraint và index đặc biệt: kiểm tra hỗ trợ và viết migration thủ công khi cần.
- Enum mới — thêm cả giá trị cũ trước khi deploy code mới.
- Large table ALTER — production dùng expand-migrate-contract.
- Đổi FK bảng có data — chuẩn bị data migration song song.

Session sống bao lâu, connection pool hoạt động ra sao?

APP-CONTEXT-SCOPED SESSION — thường một context mỗi request



Flask-SQLAlchemy scope db.session theo application context; session được remove khi context kết thúc.

SQLALCHEMY_ENGINE_OPTIONS

```
SQLALCHEMY_ENGINE_OPTIONS = {
    "pool_size": 10,
    "max_overflow": 20,  # burst
    "pool_timeout": 30,
    "pool_recycle": 1800,
    "pool_pre_ping": True,
}
```

- pool_size = persistent conn
- max_overflow = burst khi tải cao
- pool_pre_ping = kiểm tra connection khi checkout, dùng cơ chế theo dialect

api/products.py — POST có try/except

```
from ..errors import ConflictError
def create_product(body):
    p = Product(**body)
    try:
        db.session.add(p); db.session.commit()
    except IntegrityError:
        db.session.rollback()  # quan trọng!
        raise ConflictError("sku exists")
    return p, 201
```

CẢNH BÁO

- Không commit sau mỗi query — commit *một lần* sau cụm thay đổi logic.
- Không giữ session qua request — chỉ dùng `db.session` trong scope handler.
- Không quên rollback trên exception.

THỰC HÀNH 01

Lab 1 — xây dựng Product API CRUD hoàn chỉnh

Bài tập chính buổi học — viết service Flask có 6 endpoint, tích hợp SQLite/PostgreSQL, sử dụng application factory, blueprint, marshmallow schema và SQLAlchemy 2.x.

ENDPOINTS CẦN TRIỂN KHAI

GET	`/api/v1/products`	Lấy danh sách có phân trang — query `?page=1&size=20`
GET	`/api/v1/products/<int:id>`	Lấy 1 sản phẩm — 404 nếu không thấy
POST	`/api/v1/products`	Tạo sản phẩm — body JSON; 409 nếu sku trùng
POST	`/api/v1/products/<int:id>/reserve`	Đặt trước *1 sản phẩm* — body: `{"qty": 1}`; 409 nếu hết hàng
PATCH	`/api/v1/products/<int:id>`	Cập nhật một phần — chỉ field có trong body mới đổi
DELETE	`/api/v1/products/<int:id>`	Xóa sản phẩm — soft-delete (đánh dấu archived=true)

YÊU CẦU & BẤT BIẾN

Schema sản phẩm
sku (3-32 ký tự), name (1-255), price_cents (≥0), qty_in_stock (mặc định 0).

Phân trang
Response: `{"items": [...], "page": 1, "size": 20, "total": N}`.

Reserve
Giảm `qty_in_stock`; không cho < 0.

Soft delete
List/get mặc định loại trừ `archived=True`.

Xử lý lỗi
404/409/422 có JSON `{"error": "...", "detail": "..."}`.

THỰC HÀNH 01 · 1/2

Bước 1 — khai báo Model và 4 Schema cho Product

models/product.py

```
from datetime import datetime
from typing import Optional
from sqlalchemy import String, Integer, Boolean, DateTime, func
from sqlalchemy.orm import Mapped, mapped_column
from ..extensions import db

class Product(db.Model):
    __tablename__ = "products"
    id: Mapped[int] = mapped_column(primary_key=True)
    sku: Mapped[str] = mapped_column(
        String(32), unique=True, index=True, nullable=False)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[Optional[str]] = mapped_column(String(2000))
    price_cents: Mapped[int] = mapped_column(Integer, nullable=False)
    qty_in_stock: Mapped[int] = mapped_column(Integer, default=0)
    archived: Mapped[bool] = mapped_column(
        Boolean, default=False, index=True)
    created_at: Mapped[datetime] = mapped_column(
        DateTime, server_default=func.now(), nullable=False)
```

THỰC HÀNH 01 · 2/2

Bước 1 — khai báo Model và 4 Schema cho Product

schemas.py — 4 schema

```
from marshmallow import Schema, fields, validate
# ——— output ———
class ProductOut(Schema):
    id = fields.Int()
    sku = fields.Str()
    name = fields.Str()
    description = fields.Str(allow_none=True)
    price_cents = fields.Int()
    qty_in_stock = fields.Int()
    archived = fields.Bool()
# ——— input tạo mới ———
class ProductIn(Schema):
    sku = fields.Str(required=True,
        validate=validate.Length(min=3, max=32))
    name = fields.Str(required=True,
        validate=validate.Length(min=1, max=255))
        validate=validate.Range(min=0))
    qty_in_stock = fields.Int(load_default=0,
        validate=validate.Range(min=0))
class ProductPatch(Schema):
    name = fields.Str()
    price_cents = fields.Int(
        validate=validate.Range(min=0))
# ——— /reserve ———
class ReserveIn(Schema):
    qty = fields.Int(required=True,
        validate=validate.Range(min=1))
```

Bước 2 — Blueprint Resource xử lý 6 endpoint

api/products.py — toàn bộ resource

```
from flask import Blueprint, jsonify, request
from sqlalchemy.exc import IntegrityError
from ..extensions import db
from ..models import Product
from ..errors import Conflict
from ..schemas import (
    ProductOut, ProductIn, ProductPatch, ReserveIn)
products_bp = Blueprint(
    "products", __name__, url_prefix="/api/v1/products")
@products_bp.get("")
def list_products():
    page = max(1, int(request.args.get("page", 1)))
    size = min(100, max(1,
        int(request.args.get("size", 20))))
    base = db.select(Product).where(
        Product.archived.is_(False))
    rows = db.session.execute(
        base.order_by(Product.id)
        .offset((page-1) * size).limit(size)
    ).scalars().all()
    return jsonify({
        "items": [ProductOut().dump(p) for p in rows],
        "page": page, "size": size,
    }), 200
        "total": db.session.scalar(
            db.select(db.func.count()).select_from(Product)
            .where(Product.archived.is_(False))),
```

Sinh viên tự thêm GET /<int:id> và POST /<int:id>/reserve theo mẫu. Commit lên git push lên origin để nộp.

THỰC HÀNH 01 · 2/2

Bước 2 — Blueprint Resource xử lý 6 endpoint

Tiếp theo (cùng file)

```
@products_bp.post("")
def create_product():
    body = ProductIn().load(request.json)
    p = Product(**body)
    db.session.add(p)
    try:
        db.session.commit()
    except IntegrityError:
        db.session.rollback()
        raise Conflict(message="sku already exists")
    return ProductOut().dump(p), 201

@products_bp.patch("/<int:id>")
def patch_product(id):
    body = ProductPatch().load(
        request.json, partial=True)
    p = db.get_or_404(Product, id)
    for k, v in body.items():
        setattr(p, k, v)
    db.session.commit()
    return ProductOut().dump(p), 200

@products_bp.delete("/<int:id>")
def delete_product(id):
    p = db.get_or_404(Product, id)
    p.archived = True
    db.session.commit()
    return "", 204
```

- *Sinh viên tự thêm `GET /<int:id>` và `POST /<int:id>/reserve` theo mẫu.*
- *Commit lên git push lên origin để nộp.*

Trả lỗi HTTP đúng cách — 4xx có nghĩa, 5xx có log

STATUS	Ý NGHĨA	KHI NÀO DÙNG	VÍ DỤ
400	Bad Request — input không parse được	JSON sai cú pháp hoặc content-type sai	request.body = "{ bad json"
404	Not Found — không tìm thấy tài nguyên	GET /products/9999 không tồn tại	db.get_or_404(Product, id)
409	Conflict — xung đột với trạng thái hiện tại	unique constraint, version mismatch	sku đã tồn tại
422	Unprocessable — cú pháp OK, nội dung sai	Marshmallow validate fail	flask-smorest tự trả 422

X ĐỪNG TRẢ 200 + body có chữ "error"

HTTP status code là phần *hợp đồng* quan trọng nhất. Client (mobile/web) phân nhánh hành vi theo status — không phải theo một trường `status` trong body. Một response lỗi PHẢI có:

- status code thuộc 4xx (lỗi client) hoặc 5xx (lỗi server)
- body JSON có code (string machine-readable) và message (chuỗi hiển thị)
- header Nếu dùng Problem Details: application/problem+json (RFC 9457)

Exception riêng và global error handler

api/errors.py

```
from flask import jsonify
from werkzeug.exceptions import HTTPException
class ApiError(Exception):
    """Base — mọi lỗi business kế thừa class này."""
    status_code = 500
    code = "internal_error"
    def __init__(self, code=None, message="", status=None):
        super().__init__(message)
        if code: self.code = code
        if status: self.status_code = status
        self.message = message
class NotFound(ApiError):
    status_code = 404; code = "not_found"
class Conflict(ApiError):
    status_code = 409; code = "conflict"
class OutOfStock(ApiError):
    status_code = 409; code = "out_of_stock"
```

RAISE Ở ĐÂU

Resource layer

```
if product.qty_in_stock < qty:
    raise OutOfStock(
        message=f"only
{product.qty_in_stock} left")
```

Repo layer — trả None, không raise

```
def get_by_id(id):
    return db.session.get(Product,
        id)
    # None nếu không có
```

Repo là nơi test nhiều nhất — không ép biết Flask HTTPException.

Phân biệt 422 vs 409 vs 500

422 validate fail · **409** state conflict · **500** chưa lường.

Exception riêng và global error handler

api/errors.py — global error handlers

```
# ——— register handler trong create_app() ———
def register_error_handlers(app):
    @app.errorhandler(ApiError)
    def _on_api(err):
        return jsonify({"code": err.code,
            "message": err.message}), err.status_code
    @app.errorhandler(HTTPException)
    def _on_http(err):
        return jsonify({
            "code": err.name.lower().replace(" ", "_"),
            "message": err.description}), err.code
    @app.errorhandler(Exception)
    def _on_err(err):
        app.logger.exception("unhandled")
        return jsonify({"code": "internal_error"}), 500
```

Schema validate không chỉ là "check có field"

schemas.py — validate sâu

```
from marshmallow import (
    Schema, fields, validate, validates, ValidationError)
class ProductIn(Schema):
    # field-level: list các rule
    sku = fields.Str(required=True, validate=[
        validate.Length(min=3, max=32),
        validate.Regexp(r"^[A-Z0-9-]+$"),
    ])
    name = fields.Str(required=True,
        validate=validate.Length(max=255))
    price_cents = fields.Int(required=True,
        validate=validate.Range(min=0, max=10_000_000))
    # Validate từng field
    @validates("name")
    def _name(self, value, **kwargs):
        if value.strip() == "":
            raise ValidationError("name trống")
        # Không dùng return để normalize

    @validates("price_cents")
    def _price(self, value, **kw):
        # Kiểm tra miền giá hợp lệ
        if value < 0:
            raise ValidationError(
                "giá không hợp lệ")
```

KHI NÀO DÙNG GÌ

- ▸fields-levelvalidate= cho rule tái sử dụng.
- ▸@validates kiểm tra 1 field; @pre_load để normalize input.
- ▸@validates_schema cho quy tắc liên trường.
- ▸@post_load convert dict → domain cuối cùng.

ĐỪNG LÀM

- ✗ Gọi DB trong @validates — schema thuần, không side-effect.
- ✗ Validate *sau khi* save — DB đã có data xấu.
- ✗ Trả 200 + message trong body khi validate fail.

Cấu hình 12-factor — env vars + defaults hợp lý

config.py — class hierarchy

```python
import os
class DefaultConfig:
    SECRET_KEY = os.environ.get(
        "SECRET_KEY", "dev-not-for-prod")
    JSON_SORT_KEYS = False
    SQLALCHEMY_DATABASE_URI = os.environ.get(
        "DB_URL", "sqlite:///dev.db")
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    SQLALCHEMY_ENGINE_OPTIONS = {
        "pool_pre_ping": True, "pool_recycle": 1800}
    LOG_LEVEL = os.environ.get("LOG_LEVEL", "INFO")
class TestConfig(DefaultConfig):
    TESTING = True
    SQLALCHEMY_DATABASE_URI = "sqlite:///:memory:"
    LOG_LEVEL = "WARNING"
class ProductionConfig(DefaultConfig):
    LOG_LEVEL = "WARNING"
    SQLALCHEMY_ENGINE_OPTIONS = {
        "pool_size": 10, "max_overflow": 20,
        "pool_recycle": 1800, "pool_pre_ping": True}
# ——— factory.load ———
def create_app(config=None):
    cfg = config or os.environ.get(
        "APP_CONFIG", DefaultConfig)
    app = Flask(__name__)
    app.config.from_object(cfg)
    app.config.from_prefixed_env("APP")
```

.env.example — commit lên git

```
# DB
DB_URL=postgresql+psycopg://user:pwd@db/app
# Flask
SECRET_KEY=<tạo-bằng-secrets-token-urlsafe-32>
APP_CONFIG=product_api.config.ProductionConfig
# Log
LOG_LEVEL=INFO
```

BA QUY TẮC

1. Defaults không hard-code logic

Đổi database cần kiểm tra driver, dialect và migration.

2. Secret không commit lên git

.env.example có placeholder; bí mật thật qua Vault/SSM/K8s.

3. Cấu hình từ môi trường, thường cần restart process

Mỗi config qua `os.environ`; field mới chỉ cần new code đọc.

Log một dòng JSON — query được, alert được

log xấu — string

```
[2026-08-15 10:23] INFO views.create_product
user=vna prod=iphone-15 145ms trace=abc
```

✕ Khó grep; không query *"request lỗi >1s"*.

log tốt — JSON

```
{"ts": "2026-08-15T10:23:45Z", "level": "INFO",
 "event": "done", "request_id": "abc",
 "status": 201, "ms": 12}
```

✓ Loki/ELK query theo field; alert áp nhiều service.

LỖI PHỔ BIẾN

- ✗ `print()` thay app.logger.
- ✗ String dài 1KB — tách field.
- ✗ Log password, token, secret.
- ✗ DEBUG trong production.

Log một dòng JSON — query được, alert được

logging_config.py — stdlib dictConfig

```
import json, logging, sys, time
from flask import has_request_context, request
class JsonFormatter(logging.Formatter):
    def format(self, rec):
        p = {"ts": self.formatTime(rec,
             "%Y-%m-%dT%H:%M:%SZ"),
             "level": rec.levelname, "msg": rec.getMessage()}
        if has_request_context():
            p["rid"] = request.headers.get("X-Request-Id", "-")
        return json.dumps(p, ensure_ascii=False)

def configure(app):
    h = logging.StreamHandler(sys.stdout)
    h.setFormatter(JsonFormatter())
    app.logger.handlers = [h]  # thay handler mặc định
    app.logger.setLevel(app.config["LOG_LEVEL"])
    # đo latency
    @app.before_request
    def _t0():
        request._t0 = time.perf_counter()
    @app.after_request
    def _done(resp):
        ms = int((time.perf_counter() - request._t0) * 1000)
        app.logger.info("done",
            extra={"s": resp.status_code, "ms": ms})
        return resp
```

LỖI PHỔ BIẾN

- ✗ `print()` thay app.logger.
- ✗ String dài 1KB — tách field.
- ✗ Log password, token, secret.
- ✗ DEBUG trong production.

Liveness, readiness — hai probe khác nhau

/healthz — LIVENESS

"Tôi còn sống không?"

Probe nhẹ, *không* phụ thuộc DB. Trả **200** nếu process còn chạy; restart pod nếu fail.

/ready — READINESS

"Tôi nhận traffic được chưa?"

Kiểm tra dependency (DB, cache). Trả **503** nếu chưa sẵn sàng; orchestrator *không restart*.

sai lầm hay gặp

- Gộp 2 endpoint thành 1.
- /ready gọi external API không cache.
- Restart khi liveness fail mà không xem log.

Liveness, readiness — hai probe khác nhau

api/health.py

```python
from flask import Blueprint, jsonify
from sqlalchemy import text
from ..extensions import db
health_bp = Blueprint("health", __name__, url_prefix="")
# liveness — không DB
@health_bp.get("/healthz")
def liveness():
    return jsonify({"status": "ok"}), 200
# readiness — check DB
@health_bp.get("/ready")
def readiness():
    checks = {}
    try:
        db.session.execute(text("SELECT 1"))
        checks["db"] = "ok"
    except Exception:
        checks["db"] = "err"
    ok = all(v == "ok" for v in checks.values())
    return jsonify({"status": "ok" if ok else "degraded",
        "checks": checks}, 200 if ok else 503)
```

cấu hình K8s mẫu

```yaml
livenessProbe:
  httpGet: { path: /healthz, port: 5000 }
  periodSeconds: 10
  failureThreshold: 3

readinessProbe:
  httpGet: { path: /ready, port: 5000 }
  periodSeconds: 5
  failureThreshold: 2
```

sai lầm hay gặp

- Gộp 2 endpoint thành 1.
- /ready gọi external API không cache.
- Restart khi liveness fail mà không xem log.

Tại sao `flask run` không phải production

FLASK DEV SERVER

✗ Dev server không dành cho yêu cầu vận hành production.

✗ Không graceful reload, không restart khi crash.

✗ Debug bật có thể lộ source.

✗ Không signal handling — Ctrl+C mới thoát.

GUNICORN

√ Pre-fork — 1 master + N worker.

√SIGTERM graceful shutdown.

√ Có cơ chế quản lý và khởi động lại worker khi cần.

√ WSGI server được sử dụng rộng rãi; kiểm tra release hiện hành.

Nginx: TLS + gzip + rate limit. Gunicorn chỉ lo WSGI.

Tại sao `flask run` không phải production

wsgi.py + gunicorn.conf.py

```
# wsgi.py — entry point
from product_api.factory import create_app
app = create_app()
# gunicorn.conf.py
bind = "0.0.0.0:5000"
workers = 2  # điều chỉnh theo tải và giới hạn CPU
worker_class = "sync"  # hoặc "gthread"
timeout = 60  # worker timeout
graceful_timeout = 30
keepalive = 5  # sync worker không dùng keepalive
preload_app = False  # cân nhắc khi khởi tạo kết nối
accesslog = "-"
errorlog = "-"
loglevel = "info"
proc_name = "product-api"
# chạy production:
$ gunicorn -c gunicorn.conf.py wsgi:app
```

nginx phía trước Gunicorn

```
server {
    listen 80;
    server_name api.example.com;
    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_read_timeout 30s;
    }
}
```

Nginx: TLS + gzip + rate limit. Gunicorn chỉ lo WSGI.

Dockerfile multi-stage — tách builder và runtime

Dockerfile (multi-stage)

```
# ——— stage 1: builder ———
FROM python:3.12-slim AS builder
WORKDIR /build
COPY pyproject.toml ./
COPY src ./src
RUN pip install --no-cache-dir uv \
    && uv venv /opt/venv
ENV VIRTUAL_ENV=/opt/venv \
    PATH=/opt/venv/bin:$PATH
RUN uv pip install --no-cache .
# src đã được copy ở bước trên
# package đã được cài trong venv
# ——— stage 2: runtime ———
FROM python:3.12-slim
WORKDIR /app
RUN groupadd -r app && \
    useradd -r -g app -d /app app
# Copy venv từ builder
    /opt/venv /opt/venv
COPY --from=builder --chown=app:app \
# Source nằm trong package đã cài
ENV VIRTUAL_ENV=/opt/venv \
    PATH=/opt/venv/bin:$PATH \
    PYTHONUNBUFFERED=1
USER app
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "product_api.factory:create_app()"]
```

Dockerfile multi-stage — tách builder và runtime

docker-compose.yml — dev stack

```
services:
  db:
    image: postgres:16-alpine
    environment: { POSTGRES_USER: app, POSTGRES_PASSWORD: devpw,
POSTGRES_DB: appdb }
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app -d appdb"]
      interval: 5s
      retries: 10
    volumes: [pgdata:/var/lib/postgresql/data]
  api:
    build: .
    environment:
      DB_URL: postgresql+psycopg://app:devpw@db/appdb
    depends_on:
      db: { condition: service_healthy }
    ports: ["5000:5000"]
    # Bind mount source chỉ dùng khi đã cấu hình dev
volumes: { pgdata: {} }
```

TỐI ƯU

- Multi-stage có thể giảm kích thước; cần đo image thực tế.
- `--no-cache-dir`; `--no-install-recommends`.
- Không chạy root — dùng USER app.
- `condition: service_healthy` chờ DB sẵn sàng.

TỔNG KẾT

Bài tập

1. Hoàn thiện Lab 1 — đẩy code lên repo cá nhân.

2. Thêm một resource OrderItem liên kết Order và Product (tránh gán một Product duy nhất cho Order); resource này dùng blueprint riêng, /api/v1/orders.

3. Đóng gói toàn bộ service bằng Dockerfile + docker-compose; chạy được trên máy khác mà không sửa gì.

4. Thêm /healthz + /ready; *không* gộp chung.

5. Viết pytest cho ít nhất 1 endpoint POST và 1 endpoint GET có eager loading — khuyến khích kiểm tra số truy vấn để phát hiện N+1.

TÀI LIỆU THAM KHẢO

Connexion docs — connexion.readthedocs.io

flask-smorest docs — flask-smorest.readthedocs.io

SQLAlchemy 2.x tutorial — doc.sqlalchemy.org

Flask-Migrate (Alembic) — flask-migrate.readthedocs.io

Gunicorn settings — docs.gunicorn.org

BUỔI TỚI — BUỔI 08

API Testing & Quality Assurance

Test strategy · Postman/Newman · contract testing · load test với k6 · OpenAPI conformance.